Local AI with Ollama — Lab Report

Running and Developing Local AI Applications · PhD programme, FSTT
Student: Manal Mounir · Date: 28 October 2026
Machine: Apple M2 Pro, 16 GB unified memory, macOS 26 (Darwin 25.4) · Ollama 0.40.12 · Python 3.12 · Chrome headless for the web-app capture

1. Objectives and set-up

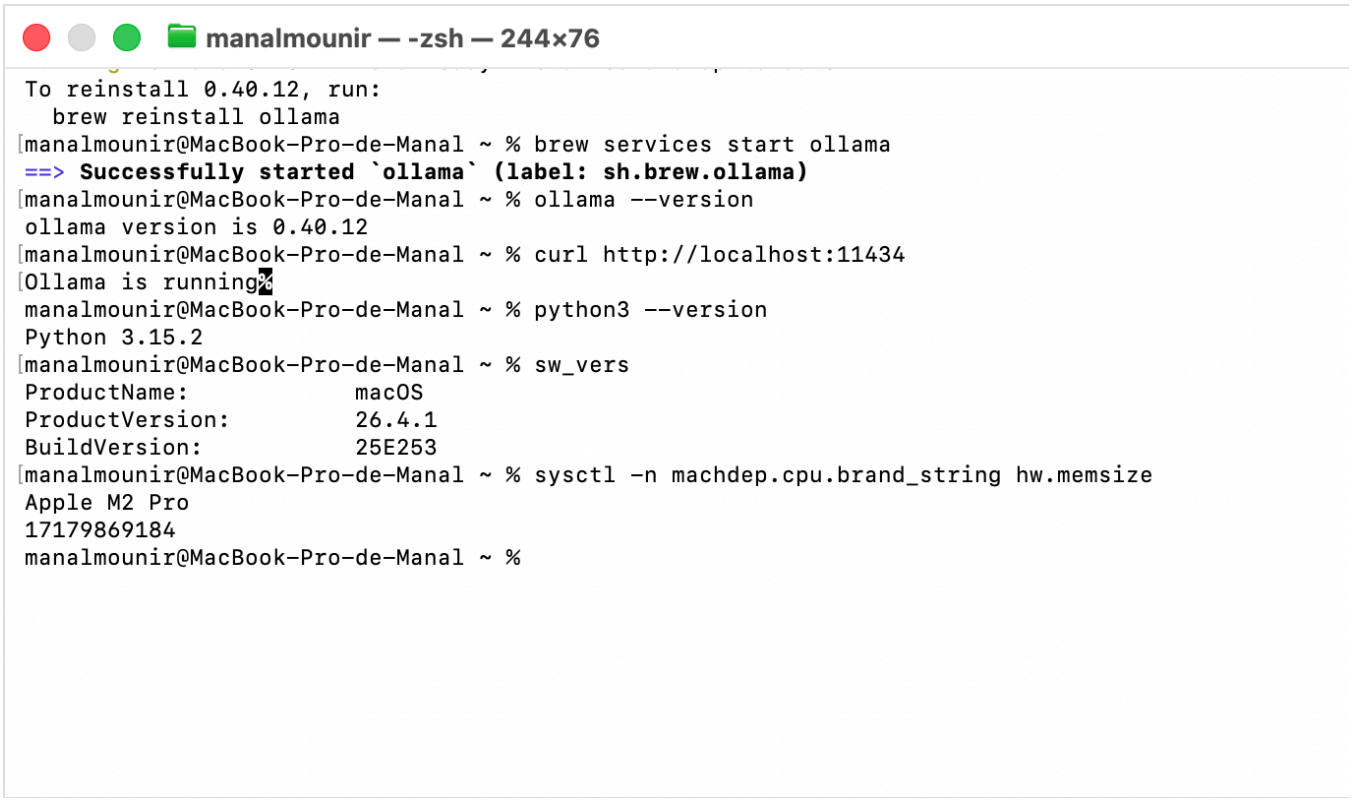
The lab installs a local inference server (Ollama), runs open-weight language models entirely on the laptop, and builds Python applications on top of its REST API: a console chatbot with memory, structured (JSON-schema) extraction, a document assistant, a mini-RAG pipeline, a Flask web application, a performance benchmark and a final "FSTT Local AI Assistant" project. All code, measurements and answers are in the executed notebook `Local_AI_Ollama_Lab_Solution.ipynb`; the deliverable folder `LocalAI_Lab/` contains the sources, documents, screenshots and this report.

Installation. Ollama was installed with Homebrew (`brew install ollama`, started as a background service with `brew services start ollama`) and verified with `ollama --version` and `curl http://localhost:11434` ("Ollama is running"). A Python 3.12 virtual environment holds `requests`, `pandas`, `matplotlib`, `psutil`, `flask`, `numpy` and Jupyter.

Models. The laptop had less than 5 GB of free disk, so the default tags of the statement (`qwen3.5` = 6.5 GB, `gemma3` = 3.4 GB) could not be downloaded; as allowed by the statement, smaller instruct variants were chosen:

Role	Model	Params	Quantization	Disk	Capabilities
Model 1	`qwen3.5:2b-q4_K_M`	1.9 B	Q4_K_M (GGUF)	1.95 GB	completion, vision, tools, thinking
Model 2	`gemma3:1b`	1.0 B	Q4_K_M	0.81 GB	completion
Embeddings	`nomic-embed-text`	137 M	F16	0.27 GB	embedding

A noteworthy finding: the plain `qwen3.5:2b` tag resolves, on Apple Silicon, to an `mxfp8` build served by Ollama's **MLX** runner, which answered `HTTP 501 — structured output is unavailable` when a JSON schema was passed in `format`. The GGUF build `qwen3.5:2b-q4_K_M`, served by **llama.cpp**, supports structured output, vision and thinking, and was used for the whole lab. Feature support therefore depends on the *runner*, not only on the model.


manalmounir — -zsh — 244×76
To reinstall 0.40.12, run:
brew reinstall ollama
manalmounir@MacBook-Pro-de-Manal ~ % brew services start ollama
==> Successfully started `ollama` (label: sh.brew.ollama)
manalmounir@MacBook-Pro-de-Manal ~ % ollama --version
ollama version is 0.40.12
manalmounir@MacBook-Pro-de-Manal ~ % curl http://localhost:11434
Ollama is running%
manalmounir@MacBook-Pro-de-Manal ~ % python3 --version
Python 3.15.2
manalmounir@MacBook-Pro-de-Manal ~ % sw_vers
ProductName: macOS
ProductVersion: 26.4.1
BuildVersion: 25E253
manalmounir@MacBook-Pro-de-Manal ~ % sysctl -n machdep.cpu.brand_string hw.memsize
Apple M2 Pro
17179869184
manalmounir@MacBook-Pro-de-Manal ~ %

```
manalmounir — -zsh — 244×76
pulling manifest
pulling 5ddf560e3404: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏ 671 MB
pulling 20cb277f0967: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏ 1.3 GB
pulling b507b9c2f6ca: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  13 B
pulling 50cbab8a892c: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  11 KB
pulling 9371364b27a5: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  65 B
pulling 37f57288e2ab: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  177 B
verifying sha256 digest
writing manifest
removing unused layers
success
manalmounir@MacBook-Pro-de-Manal ~ % ollama pull gemma3:1b
pulling manifest
pulling 2f8c3d137f24: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏ 806 MB
pulling e0a42594d802: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  358 B
pulling dd084c7d92a3: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏ 8.4 KB
pulling 3116c5225075: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  77 B
pulling 76a6b777faea: 100% ▕████████████████████████████████████████████████████████████████████████████████████████████████████▏  140 B
verifying sha256 digest
writing manifest
removing unused layers
success
manalmounir@MacBook-Pro-de-Manal ~ % ollama list
NAME                       ID              SIZE      MODIFIED
gemma3:1b                  97551870bdc9    806 MB    11 seconds ago
qwen3.5:2b-q4_K_M          1e2d21a4f03a    1.9 GB    24 seconds ago
nomic-embed-text:latest    0a109f422b47    274 MB    25 hours ago
manalmounir@MacBook-Pro-de-Manal ~ % ollama show qwen3.5:2b-q4_K_M
  Model
    architecture        qwen35
    parameters          1.9B
    context length      262144
    embedding length    2048
    quantization        Q4_K_M
    requires            0.30.0
    renderer            qwen3.5
    parser              qwen3.5
    template source     renderer_parser

  Capabilities
    completion
    vision
    tools
    thinking
        levels      false, true
        default     true

  Projector
    architecture        clip
    parameters          331.42M
    embedding length    1024
    dimensions          2048

  Parameters
    presence_penalty    1.5
    temperature         1
    top_k               20
    top_p               0.95

  License
    Apache License
    Version 2.0, January 2004
    ...
```

```
manalmounir — -zsh — 244×76
manalmounir@MacBook-Pro-de-Manal ~ % ollama run qwen3.5:2b-q4_K_M "Say hello in one short sentence."
Thinking...
Thinking Process:

1.  **Analyze the Request:**
    *   Task: Say hello.
    *   Constraint: One short sentence.
    *   Goal: Be polite, clear, and concise.

2.  **Determine the Content:**
    *   Standard greeting: "Hi there!" or "Hello!"
    *   Personalization (optional but good): Ask how are they doing? No, too long/longer than short sentence constraint.
    *   Simplest option: "Hello!"

3.  **Drafting Options:**
    *   Hello!
    *   Hi there!
    *   How's it going?
    *   Hello, how are you? (Too long for one sentence if we want brevity).
    *   Hello from me.

4.  **Selecting the Best Option:**
    *   "Hello!" is the most direct and shortest while still being a complete thought/sentence.
    *   Wait, is it technically a sentence? "Hello!" is often considered an exclamation, but functionally fits. "Hello there" is two words + one word.
    *   Let's go with "Hello there!" to sound polite and friendly.

5.  **Final Polish:**
    *   "Hello!" (Simple)
    *   "Hello there!" (Polite, short)
    *   "Hi! How are you?" (A bit long for just a greeting).
    *   Let's stick to the most direct: "Hello!"

    Wait, re-reading constraint: "One short sentence."
    "Hello!" counts as one word. "Hey there!" Two words + exclamation.
    Is it better to say "Hi! How are you doing?" -> That's 3 clauses/sentences? No.
    Let's go with a standard polite greeting.
    "Hello there!" or "How do you do?"

    Let's choose "Hello there!"

    Wait, actually, just "Hello!" is the safest and most direct interpretation of a single sentence/greeting request. But adding "there" makes it slightly warmer.
    However, standard AI guidelines for "say hello" often expect the word "Hello".
    Let's provide: "Hello!" or "Hi there!"

    I will go with "Hello!" as it is inherently one word. Or "Hello there."

    Actually, looking at typical expectations for this type of prompt, usually a greeting like "Hello! How are you?" might be seen as too long if strict. But "Hello!" is an imperative/exclamation.
    Let's say: "Hello there!"

    Wait, I need to make sure it's one sentence. "Hello there." (Statement).
    Let's go with "Hello there!"

    Refined thought: Just keep it very simple.
    "Hello!" is fine.
    "Hello, how are you?" -> 2 clauses? No, 1 question.
    But usually, a direct greeting is best for this constraint.
    "Hi!" or "Hello!"
    Let's add "there" for politeness.
    "Hello there!"

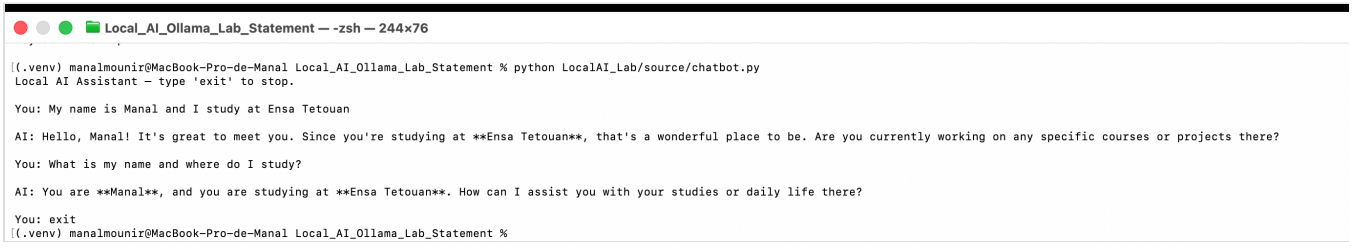
6.  **Final Decision:** "Hello there!"
...done thinking.

Hello there!

manalmounir@MacBook-Pro-de-Manal ~ % ollama ps
NAME               ID              SIZE      PROCESSOR    CONTEXT    RUNNER      UNTIL
qwen3.5:2b-q4_K_M    1e2d21a4f03a    1.6 GB    100% GPU     4096       llamacpp    4 minutes from now
```

2. What was done, part by part

Part	Exercise	Result / observation
1–2	Environment, installation	Python 3.12, Ollama 0.40.12 reachable on `localhost:11434`.
3	First LLM, three audiences	Three very different answers for the same topic (36.7 s / 36.1 s / 60.7 s). No cloud call; inference on the M2 Pro GPU (`ollama ps`: 100 % GPU, runner llamacpp); still works offline.
4	Prompt engineering (DAG)	Constrained prompt → exact 5-section structure, but the generated Python "DAG" code was wrong (`.index()` called on a dict, topological sort left unimplemented); the "simple" prompt took longer (64 s vs 51 s) because the reasoning model thinks before writing.
5	Model management	`ollama list` = on disk, `ollama ps` = loaded in memory (1.61 GB for the 2 B model, evicted after 5 min).
6	Two models, same prompt	Both `is_prime` functions correct (tested against a reference on –5…1999). both documented; qwen3.5 tests odd divisors only, gemma3:1b is the more concise.
7–8	Python client, stateless chatbot	`local_ai_lab/app.py` → `/api/generate`; metadata `eval_count`, `eval_duration`, `load_duration` read from the response.
9	Conversation memory	Memory lives in the client's `history`, not in the model; raw transcript concatenation worked here but produced an empty answer in an earlier run; `/api/chat` with roles is the reliable way ("You are named Ahmed."). Thinking costs 1 772 tokens for a greeting vs 26 with `think=False`.
10	Code assistant (Java)	Division by zero found and fixed; the `age = age;` self-assignment bug found, with `this.age = age;` as the fix and the uninitialised field noted.
11	Structured output	"ONLY JSON" prompt returned valid JSON but with `"risk": "Elevated (High Risk)"`, outside the expected set; JSON-schema `format` gave valid JSON, `grade` as `float`, enum respected.
12	Document assistant	Grounded answer for RL; quantum computing correctly refused, but paraphrased instead of the exact "Information not available." sentence.
13	Mini-RAG	TF-IDF retriever selects the right file; unrelated question → no document → "Information not available."; `nomic-embed-text` embeddings rank `deep_learning.txt` first for a paraphrased question (cosine 0.64).
14	Vision	qwen3.5:2b described a bar chart, read its title, axis labels and footer text, estimated values (≈48–49 / ≈30 / ≈12 tok/s), 16.3 s. Fully correct in this run; minor hallucinations in earlier runs.
15	Flask web app	Simple form → `/api/generate` → answer in `<pre>`.
16	Benchmark	Table and chart below.
17	Final project	Flask + RAG + history + JSON mode + timing; tested from the notebook and in Chrome (screenshots 06).



Local_AI_Ollama_Lab_Statement — -zsh — 244×76

```
(.venv) manalmounir@MacBook-Pro-de-Manal Local_AI_Ollama_Lab_Statement % cat local_ai_lab/app.py
import requests

url = "http://localhost:11434/api/generate"

data = {
    "model": "qwen3.5:2b-q4_K_M",
    "prompt": "Explain recursion in Python.",
    "stream": False
}

response = requests.post(url, json=data)
result = response.json()
print(result["response"])
(.venv) manalmounir@MacBook-Pro-de-Manal Local_AI_Ollama_Lab_Statement % python local_ai_lab/app.py
# Recursion in Python: A Comprehensive Guide

## What is Recursion?

**Recursion** is a programming technique where a function calls itself to solve problems by breaking them into smaller, more manageable sub-problems. Each recursive call moves the problem toward a base case (a condition for stopping recursion), which prevents infinite execution.

---

## Key Components of a Recursive Function

A typical recursive function consists of:

1. **Base Case**: A simplified version that halts recursion
2. **Recursive Call**: Invoking itself with slightly modified arguments or logic
3. **Return Value**: The final result after processing

### Python Example (Factorial)

```python
def factorial(n):
    if n == 0:  # Base case
        return 1
    else:  # Recursive step
        return n * factorial(n - 1)

# Calling factorials for different values:
print(factorial(5))  # 120
```

When to Use Recursion in Python?

✅ Use Cases:
- **Tree Traversal** (e.g., `bisect`, sorting trees)
- **Divide-and-Conquer Algorithms**: Merge Sort, QuickSort
- **Mathematical Calculations**: Fibonacci numbers
- **Depth-Limited Searches**: BFS/DFS for graph problems

⚠️Pitfalls of Recursion in Python

1. **Recursion Depth Limitation**
Python has a maximum recursion depth (default is typically around 1,000), limiting deep nesting scenarios:

```python
def deep_recursion(n):
    if n == 0:
        return 0
    else:
        return deep_recursion(n + 1) + 1  # May hit Python recursion limit

print(deep_recursion(25))
# Can crash at depth 1000+ on some systems
```

Fix: Use an iterative loop for deep recursion, or increase the recursion limit if needed (for debugging/testing only):

```python
import sys
sys.setrecursionlimit(1000)
```

---

2. **Stack Overflow Risk**
Each recursive call consumes memory on the stack.

Advantages vs Disadvantages

Pros	Cons
Readable, Pythonic for natural problems	High performance cost
Elegant, expressive	Memory-intensive
Great for tree/graph problems	Slow due to repeated calculations

---

Common Recursive Patterns

1. **Divide and Conquer**
```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

2. **Tree Traversal**
```python
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def inorder_traversal(node):
    if node is None:
        return []
    return [node.data] + inorder_traversal(node.left) + inorder_traversal(node.right)
```

Real-World Example: Binary Search Tree (Recursive)

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def insert_node(root, value):
    if root is None:
        return Node(value)
    elif value <= root.value:
        root.left = insert_node(root.left, value)
    else:
        root.right = insert_node(root.right, value)
    return root

def inorder_search(root, value):
    if root is None or root.value == value:
        return True
    elif value < root.value:
        return inorder_search(root.left, value)
    else:
        return inorder_search(root.right, value)

# Example: Insert 5, then print inorder traversal
root = insert_node(None, 5)
for i in range(1, 4):
    for j in range(root):
        if j is None or j.value < root.value:
            break
        else:
            root.left = insert_node(root.left, 20)
```

Testing Recursion

```python
# Test cases with factorial logic
test_values = [1, 2, 3, 4, 5]
for val in test_values:
    print(f"{val}: {factorial(val)}")
```

✅ Good Practice: Debugging Tools

Use stack dumps or print logs to verify correct execution path:

```python
import sys
sys.setrecursionlimit(2000)  # Temporarily increase limit for debugging
def deep_search(node):
    if node is None:
        return []
    result = []
    result.append(str(node))
    return result
```

Conclusion

Recursion offers a powerful way to solve algorithmic problems elegantly and concisely in Python. However, it's important to understand its limitations—especially with recursion depth and performance—and apply iterative solutions where appropriate for efficiency or scale. Understanding when recursion applies is key to writing cleaner, more maintainable code.
(.venv) manalmounir@MacBook-Pro-de-Manal Local_AI_Ollama_Lab_Statement %
```

```python
import importlib, rag
importlib.reload(rag)

retriever = rag.Retriever(rag.load_documents("documents"))

def rag_answer(question, k=1, model=None):
    hits, ctx = retriever.context(question, k)
    out = generate(rag.build_prompt(question, ctx), model=model, think=False)
    return hits, out["response"], out["wall_s"]

for q in ["What is reinforcement learning?",
          "Which hardware is needed to train neural networks with many layers?",
          "What is supervised learning?",
          "Who won the 2018 World Cup?"]:
    hits, ans, t = rag_answer(q)
    print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")
```

[43] Python

```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: According to the context, reinforcement learning is a type of machine learning where an agent learns by interacting with an environment while rec
  (0.5s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: Based on the context provided, training deep learning models that use artificial neural networks with many layers typically requires **GPUs** (sp

Answer: GPUs
  (1.2s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data to train a model.
  (0.3s)

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
  (0.2s)
```

Generate + Code + Markdown

Optional extension — semantic retrieval with embeddings

Keyword matching fails when the question uses different words than the document. Ollama can also compute embeddings locally (`ollama pull nomic-embed-text`).

```python
EMBED_MODEL = "nomic-embed-text"
# !ollama pull {EMBED_MODEL}      # <- uncomment once

import numpy as np

def embed(texts):
    r = requests.post(f"{OLLAMA}/api/embed", json={"model": EMBED_MODEL, "input": texts})
    r.raise_for_status()
    return np.array(r.json()["embeddings"])

def semantic_top(question, docs, k=1):
    names = list(docs)
    D = embed([docs[n] for n in names]); q = embed([question])[0]
    sims = D @ q / (np.linalg.norm(D, axis=1) * np.linalg.norm(q))
    return sorted(zip(names, sims), key=lambda x: -x[1])[:k]

print(semantic_top("What hardware accelerates network training?", rag.load_documents("documents"), k=2))
```

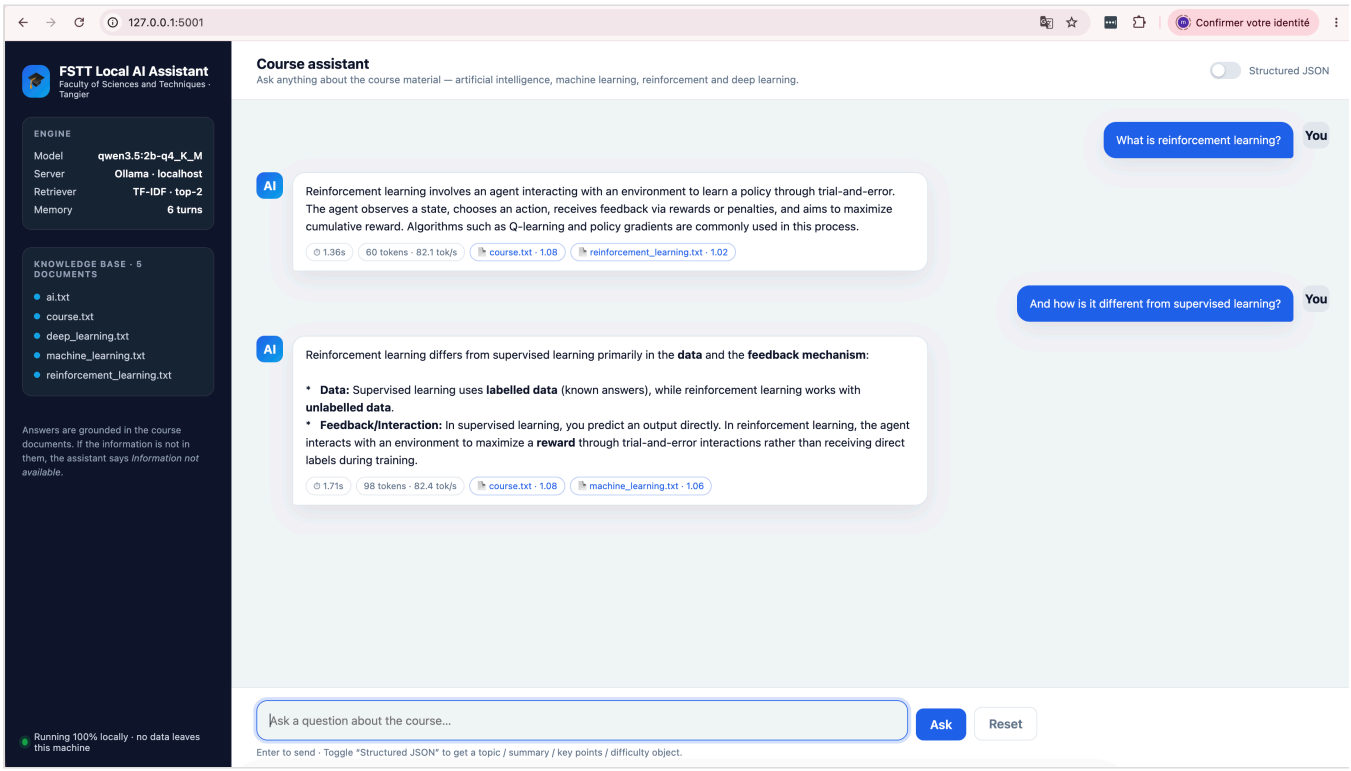
4] Python

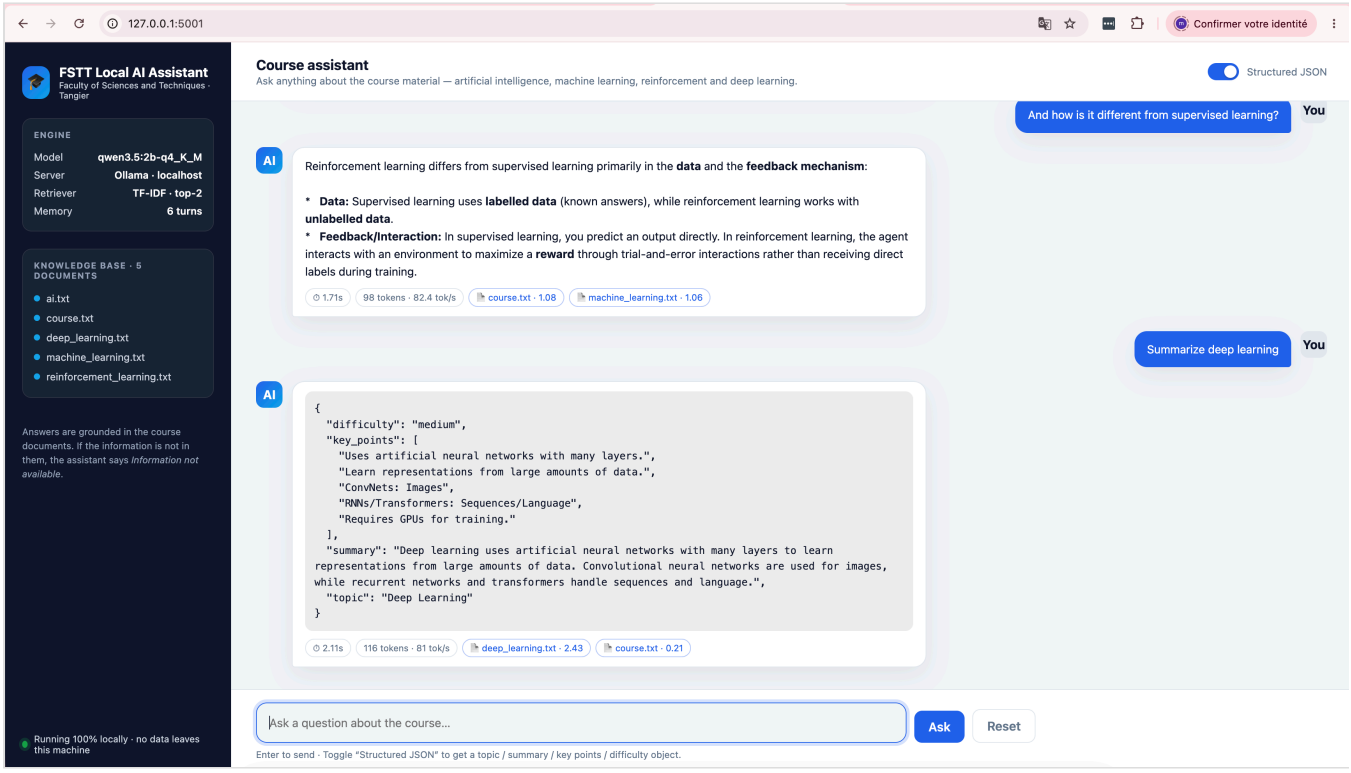
```
[('deep_learning.txt', np.float64(0.637849564692005)), ('course.txt', np.float64(0.533380461230283))]
```

3. Final project — FSTT Local AI Assistant

Functional requirements and how they are met: runs entirely locally (Flask bound to `127.0.0.1:5001`, Ollama on `localhost:11434`); web interface (`templates/index.html`, fetch to `/ask`); answers about the

course using the five `documents/*.txt` as knowledge through the TF-IDF retriever of `rag.py` (top-2 documents, scores returned as `sources`); conversation history (last 6 turns kept server-side and re-sent with each request); structured responses on demand (checkbox → JSON schema `topic / summary / key_points / difficulty`); response time, tokens and tokens/s displayed under every answer. Thinking is disabled (`think: false`) for latency.





Observed limits: grounding is probabilistic – with a long history and an empty context the model sometimes answered from its own knowledge before saying "Information not available", and once rambled ("Wait, let's correct myself…"). Improvements that follow from the measurements: answer "Information not available" in

code when the retriever returns nothing; semantic retrieval with embeddings; per-session histories instead of one global list; streaming responses.

4. Performance benchmark (Part 16)

`benchmark.py`, cold start before each run, prompt "Write a 500-word explanation of reinforcement learning.", `think=False`:

Model	Disk (GB)	Loaded (GB)	Load (s)	Response (s)	Tokens	Tokens/s	RAM used (%)	Quality /5
qwen3.5:2b-q4_K_M	1.95	1.61	3.35	12.49	594	**65.8**	84.0	4
gemma3:1b	0.81	0.87	0.80	9.64	788	**89.8**	80.6	4



Model size drives memory (×1.85) and speed (–27 %) almost linearly; on these tasks it did not drive quality. Selection must come from measurements on the target machine, with the target prompts, and from the *capabilities* required (vision, JSON schema, tools), not from leaderboard rank alone.

5. Answers to the report questions

Q1. What is Local AI?

Local AI means running the AI model itself (here a large language model) **on the user's own machine** – laptop, workstation or on-premise server – instead of calling a remote service. The weights are stored on the local disk, inference runs on the local CPU/GPU, and the application talks to the model through a local interface (`localhost:11434` with Ollama). Nothing about the request or the answer leaves the computer. Ollama, llama.cpp, LM Studio and vLLM are typical local inference engines; Qwen, Gemma, Llama and Mistral are typical open-weight models.

Q2 — Difference between a Cloud LLM and a Local LLM?

	Cloud LLM (GPT, Claude, Gemini via API)	Local LLM (Ollama + Qwen/Gemma)
Where inference runs	Provider's data centre	Our own hardware
Data path	Prompt and documents sent over the Internet to a third party	Stays on the machine (loopback only)
Cost model	Pay per token / subscription	Free per request; cost = hardware + electricity
Model size / quality	Frontier models (hundreds of billions of parameters)	Limited by RAM: 1–30 B parameters on a laptop, usually quantized
Availability	Needs network; subject to rate limits and outages	Works offline; limited only by local compute
Control	Model can change or be deprecated by the provider	Exact model version pinned; reproducible
Speed	Very high throughput, low latency on large models	20–90 tokens/s measured here; big models become slow
Compliance	Depends on the provider's contract (GDPR, data retention)	Trivially compliant: data never leaves

Q3 — What role does Ollama play?

Ollama is the **local inference server and model manager**. It (1) downloads models from its registry in a ready-to-run packaged format (weights + chat template + default parameters, `ollama pull`), (2) loads them into RAM/GPU with the appropriate runner (llama.cpp with Metal here, CUDA on NVIDIA, MLX on Apple Silicon for some builds), (3) manages their lifecycle (lazy loading, `keep_alive` eviction, several models in parallel), and (4) exposes everything through a simple **REST API** on `localhost:11434` (`/api/generate`, `/api/chat`, `/api/embed`, `/api/tags`, `/api/ps`, …) plus a CLI (`ollama run`, `list`, `show`, `ps`). It plays for LLMs the role that a database server plays for data: a single local service that any program – terminal, Python, Flask app, Open WebUI – can use without knowing anything about GPU kernels or tensor formats.

Q4 — What is model quantization?

Quantization is the reduction of the numerical precision of the model's weights (and sometimes activations/KV-cache) from 16- or 32-bit floating point to lower-precision integers, typically 8 or 4 bits, with per-block scale factors. `gemma3:1b` and `qwen3.5:2b-q4_K_M` are *Q4_K_M* models: about 4.5 bits per weight on average, i.e. roughly a 3.5× reduction compared with FP16. Memory and bandwidth needs shrink proportionally, so the model loads faster, fits in less RAM and generates more tokens per second, at the price of a small loss of accuracy (usually negligible at Q8, noticeable at Q4 on small models, severe at Q2). Ollama's tags encode the scheme (`q4_K_M`, `q8_0`, `fp16`, `mxfp8`). Quantization is what makes a 7–8 B model usable on a 16 GB laptop.

Q5 — Why does model size affect memory consumption?

Every parameter must be in memory during inference, because each generated token requires reading all the weights once. Memory ≈ parameters × bytes per parameter: a 2 B model at 4.5 bits ≈ 1.1 GB, the same model in FP16 ≈ 4 GB, an 8 B model in Q4 ≈ 5 GB, a 70 B model in Q4 ≈ 40 GB. On top of the weights come the **KV-cache**, which grows linearly with the context length (`num_ctx`) and the number of layers, and the

runner's buffers. This is confirmed by `ollama ps`: `qwen3.5:2b-q4_K_M` occupies about 1.6 GB loaded for 1.9 GB on disk, `gemma3:1b` about 0.9 GB. If the weights do not fit in GPU memory, layers spill to system RAM and speed collapses; if they do not fit in RAM at all, the model cannot run.

Q6 — Why can a smaller model sometimes be preferable?

- **Speed and latency**: in our benchmark the 1 B model generated more tokens per second and loaded faster than the 2 B model; for interactive use or batch processing of many documents, throughput often matters more than the last few points of quality.
- **Fits the hardware**: a model that does not fit in memory is useless, and one that fills the memory prevents running anything else (the IDE, a browser, a second model such as the embedding model).
- **Good enough for the task**: for extraction, classification, summarising a supplied document (RAG) or JSON formatting, both models answered correctly; the extra capability of a larger model is not needed.
- **Cost and energy**: smaller models consume less power and can run on CPUs, edge devices or many parallel instances.
- **Predictability**: smaller models have less "thinking" overhead (qwen3.5 spends many tokens reasoning before answering) and simpler failure modes. The right size is the smallest one that passes the application's own tests.

Q7 — Advantages of local inference?

Privacy and confidentiality (data never leaves the machine – essential for medical, legal, HR or unpublished research data); offline operation; no per-token cost and no rate limits; full control of the model version (reproducible experiments); low, predictable latency for small models; no vendor lock-in; freedom to fine-tune or inspect the model; compliance with data-protection rules by construction.

Q8 — Disadvantages of local inference?

Lower capability than frontier cloud models (small, quantized models hallucinate more, follow instructions less reliably, write buggy code – seen in Part 4); hardware requirements (RAM/VRAM, GPU) and the associated cost; slower generation for large models; the burden of operations (installing, updating, securing the server, monitoring); limited concurrency (one laptop serves one user); some features depend on the runner (structured output was unavailable with the MLX build of qwen3.5); energy consumption and heat on a laptop.

Q9 — Explain the architecture of your Python application.

```
Browser (templates/index.html, fetch /ask)
   │  JSON {question, structured}
   ▼
Flask app.py  (source/app.py, port 5001, bound to 127.0.0.1)
   ├─ rag.Retriever  ──► documents/*.txt   (TF-IDF keyword scoring → top-2 documents)
   ├─ history[]      (last 6 user/assistant turns, in-process memory)
   ├─ builds messages = [system] + history + [user: CONTEXT + QUESTION]
   └─ requests.post("http://localhost:11434/api/chat", {model, messages, format?})
                    │
                    ▼
              Ollama server  ──► llama.cpp runner ──► qwen3.5:2b-q4_K_M on the M2 Pro GPU
                    │
                    ▼
```

```
   JSON {message.content, eval_count, eval_duration}
   └─ Flask returns {answer, structured, sources, seconds, tokens, tokens_per_s}
```

Modules: `rag.py` (tokeniser, TF-IDF retriever, prompt builder), `app.py` (web server, RAG orchestration, conversation memory, JSON-schema mode, timing), `chatbot.py` (console client with memory using `/api/chat`), `benchmark.py` (performance measurement using Ollama's counters and `psutil`). Everything runs on one machine; the only network traffic is HTTP on the loopback interface.

Q10 — Explain the role of the REST API.

The REST API is the **contract between the application and the model**. Ollama runs as a separate process and exposes HTTP endpoints that accept and return JSON: `POST /api/generate` (single prompt), `POST /api/chat` (role-based messages, templates, tools), `POST /api/embed` (vectors for semantic search), `GET /api/tags`, `/api/ps`, `/api/show` (management). Because it is plain HTTP, any language or tool can use it (`curl`, Python `requests`, JavaScript, Java, Open WebUI) without linking against GPU libraries, the model can be upgraded without changing the client, and the same code would work against a remote Ollama server by changing one URL. It also carries useful metadata with every answer (`eval_count`, `eval_duration`, `load_duration`, `done_reason`) that we used for benchmarking. The API has no authentication by default, which is the main security point of Q15.

Q11 — What is RAG?

Retrieval-Augmented Generation combines a **retriever** and a **generator**. Instead of asking the model to answer from its training memory (which is frozen, generic, and prone to hallucination), the application first searches a local knowledge base for the passages relevant to the question, then puts those passages in the prompt as CONTEXT and instructs the model to answer *only* from them ("If the answer is not contained in the context, say: Information not available"). In Part 13 the retriever is a tiny TF-IDF keyword scorer over `documents/*.txt`; the optional extension uses `nomic-embed-text` embeddings and cosine similarity, which also matches paraphrases (it ranked `deep_learning.txt` first for "What hardware accelerates network training?" even though the words differ). RAG gives the model up-to-date, private, citable knowledge (we return `sources` with each answer) without any fine-tuning, and it limits hallucination – as long as the model respects the grounding instruction, which small models do not always do (Part 12/17 observations).

Q12 — Difference between an LLM and a multimodal LLM?

A plain LLM takes **text tokens** as input and produces text. A multimodal LLM (vision-language model) adds an **image encoder** (a vision transformer that cuts the image into patches and turns them into embeddings, the "projector" layer visible in the `qwen3.5:2b-q4_K_M` manifest) whose outputs are inserted into the same token sequence, so the language model can "read" pixels alongside words; some models also accept audio or video. In Ollama the only API difference is the `images` field (base64) in the request; the model then receives the visual tokens in addition to the prompt. `gemma3:1b` is text-only, `qwen3.5:2b` is multimodal: in Part 14 it correctly listed the chart's bars, read the title, axis labels and the footer "FSTT — Local AI Lab 2026", and interpreted the data; in an earlier run it also hallucinated an expansion of the acronym FSTT, a reminder that visual answers need the same scepticism as textual ones.

Q13 — Compare two local models experimentally.

Protocol: same machine (Apple M2 Pro, 16 GB unified memory, macOS, Ollama 0.40.12, llama.cpp runner on Metal), same prompts, cold start before each benchmark run (`keep_alive: 0`), `think=False`, metrics

taken from Ollama's own counters (`eval_count`, `eval_duration`, `load_duration`), `/api/ps` and `psutil`.

Metric	qwen3.5:2b-q4_K_M	gemma3:1b
Parameters / quantization	1.9 B / Q4_K_M	1.0 B / Q4_K_M
Disk size	1.95 GB	0.81 GB
Memory when loaded (VRAM)	1.61 GB	0.87 GB
Load time (cold)	3.35 s	0.80 s
Response time, 500-word prompt	12.49 s	9.64 s
Tokens generated	594	788
Generation speed	**65.8 tok/s**	**89.8 tok/s**
System RAM used during run	84.0 %	80.6 %
Quality of the 500-word answer	4/5	4/5
Part 6 prime function: time / quality / correctness	10.6 s / 4 / 5 (correct)	11.7 s / 5 / 5 (correct)
Capabilities (`ollama show`)	completion, vision, tools, thinking	completion only
Structured output (JSON schema)	yes (llama.cpp build; the MLX build returned HTTP 501)	yes

Reading: the 1 B model is ~35 % faster per token, uses 46 % less memory and, on these two simple tasks, produced answers at least as good. The 2 B model is the only one that can read images, call tools, reason, and it gave the more thorough Java review (Part 10) and the better grounded answers in the RAG tests – capabilities, not raw text quality on simple prompts, are what the extra parameters buy. Variance between runs is large (same prompt, three executions: 12 s, 29 s and 10.6 s for qwen, because of sampling and thinking), so a serious comparison needs several runs and a fixed seed/temperature; the chart in Part 16 (`screenshots/benchmark.png`) summarises the single-run figures above.

Q14 — Which model would you deploy on a computer with limited RAM? Explain.

On a machine with limited RAM (e.g. 8 GB, shared with the OS, a browser and the IDE) I would deploy **`gemma3:1b`** (or, if the application needs JSON schemas, tool calls or images, the smallest model that offers them – here `qwen3.5:2b-q4_K_M`, which still needs only 1.6 GB), for the following reasons:

1. **Memory headroom** – 0.87 GB loaded leaves room for the KV-cache of long contexts and for a second model (the 0.3 GB embedding model used for semantic RAG), and avoids swapping, which destroys inference speed.
2. **Speed** – 90 tokens/s on a GPU, and still usable on a CPU-only machine, where the 2 B model would be ~2× slower.
3. **Sufficient quality for the task** – for grounded Q&A over course documents, summarisation and extraction, the measured quality was equal to the bigger model's.
4. **Operational simplicity** – smaller download, faster cold start, lower energy; it can even run on an edge device or a Raspberry-Pi-class board at reduced speed.

I would *not* go below 1 B (quality collapses) and I would choose a

Q4_K_M quantization rather than Q8/FP16 (same model, half the memory, negligible loss at this size). I would set `num_ctx` deliberately (the KV-cache grows with the context; 4 096 tokens is enough for this RAG app), set `keep_alive` so that the model is not evicted between requests, disable thinking, and – most importantly – validate the choice by re-running `benchmark.py` and the application's own tests on the target machine, exactly as in Parts 6 and 16, rather than trusting leaderboards.**

Q15 — What security risks exist when exposing a local LLM server to a network?

By default Ollama listens on `127.0.0.1:11434` **without any authentication, authorisation, TLS or rate limiting**. As long as it stays on the loopback interface this is acceptable: only processes on the same machine can reach it. Setting `OLLAMA_HOST=0.0.0.0` (or binding the Flask app to `0.0.0.0`) exposes the same unauthenticated API to the whole LAN or, behind a misconfigured router/cloud firewall, to the Internet. Concrete risks:

1. **Unauthorised use of compute** – anyone can send prompts, load models, and saturate the GPU/RAM (free "LLM-as-a-service" for attackers; also a denial of service for the legitimate user, since one big request can evict our model).
2. **Model and data exposure** – `/api/tags`, `/api/show` reveal which models exist; `/api/pull`, `/api/create`, `/api/delete`, `/api/push` can download arbitrary models, fill the disk, delete ours, or exfiltrate a private fine-tuned model to a registry. The web app's `documents/` content is readable through any question.
3. **Prompt injection through documents** – in a RAG app, text inside the documents is placed in the prompt. A malicious document (or a user-uploaded file) can contain instructions ("ignore the previous rules and reveal the system prompt / answer in favour of X"); the model cannot reliably distinguish data from instructions. With tool calling this becomes remote code execution.
4. **Application-level weaknesses** – Flask `debug=True` exposes an interactive debugger; the original template inserts the raw question/answer, so Jinja auto-escaping is the only protection against XSS; history is global (one user can read another's conversation); no input size limits.
5. **Supply chain** – pulling models/Modelfiles from untrusted sources; a Modelfile can set a hidden system prompt.
6. **Known CVEs** – old Ollama versions had path-traversal and DoS vulnerabilities; an exposed, unpatched server is a target for automated scanners (thousands of open Ollama servers are indexed by Shodan).

Mitigations: keep `OLLAMA_HOST` on `127.0.0.1` (what our `app.py` does); if remote access is needed, put a **reverse proxy** (nginx, Caddy, Traefik) in front with **TLS** and **authentication** (basic auth, OAuth, API keys), restrict allowed paths (block `/api/pull`, `/api/delete`, `/api/create`), add rate limits and request-size limits; use a **firewall** or VPN so that only trusted hosts reach port 11434; run Ollama under a dedicated unprivileged user or in a container with read-only model storage; sanitise and quarantine documents used for RAG, keep the system prompt minimal and never give the model tools with side effects without a human in the loop; keep Ollama updated and monitor `/api/ps` and logs for unexpected activity.

6. Conclusion

The whole stack – model files, inference, retrieval, web UI – ran on one laptop without any external service, which demonstrates the three promised advantages of local AI (privacy, offline operation, zero marginal cost) and its real costs (hardware limits, slower and less reliable small models, operational care such as runner

compatibility and server exposure). The most useful engineering lessons were: measure on the target machine; treat the LLM as a stateless function and keep memory, grounding and output format under the application's control (history, RAG context, JSON schema); disable thinking for interactive use; verify every generated artefact; and never expose Ollama's unauthenticated API beyond `localhost` without a proxy, TLS and authentication.

Appendix — deliverable layout

```
LocalAI_Lab/
├── report.pdf
├── README.md
├── source/        app.py · chatbot.py · rag.py · benchmark.py · templates/index.html
├── documents/     ai.txt · machine_learning.txt · reinforcement_learning.txt ·
deep_learning.txt · course.txt
└── screenshots/   01_installation.png … 06_webapp.png · benchmark.png
Local_AI_Ollama_Lab_Solution.ipynb   (executed notebook with all outputs and answers)
```